

set 정리

중복 없이 값을 저장하고 자동으로 정렬해 주는 컨테이너입니다.

사용처 (Where)

- 중복 제거
- 값 존재 여부 확인
- 정렬된 상태 유지
- 삽입하면서 자동 정렬이 필요할 때

방법 (How to Use)

- insert로 추가
- find로 존재 확인
- erase로 삭제
- 순회하면 오름차순으로 나옴

기본 정보

- 필요 헤더: `#include <set>`
- 기본 선언: `set<int> s;`

왜 써야 할까? (Why)

- 중복 제거와 정렬을 동시에 해결
- 직접 정렬/중복검사 코드를 줄임
- 검색 성능이 안정적으로 좋음

주요 함수

함수/문법	의미
<code>insert(x)</code>	값 추가
<code>find(x)</code>	위치 검색
<code>count(x)</code>	존재하면 1
<code>erase(x)</code>	값 삭제
<code>lower_bound(x)</code>	x 이상 첫 위치

기본 코드 예시

```
#include <iostream>
#include <set>
using namespace std;

int main() {
    set<int> s;
    s.insert(3);
    s.insert(1);
    s.insert(3);

    for (int x : s) cout << x << ' ';
}
```

set 비교와 응용

STL을 직접 구현이나 C 스타일 코드와 비교하고, 실제 문제에서 쓰는 방식을 확인합니다.

시간 복잡도

동작	복잡도
insert/find/erase	$O(\log n)$
순회	$O(n)$
중복 저장	불가

STL vs 직접 구현 vs C 스타일

구분	STL	직접 구현	C 스타일
개발 난이도	이미 구현된 기능 사용	자료구조 설계와 예외 처리 필요	배열/포인터 직접 관리
코드 길이	짧고 읽기 쉬움	길고 버그 가능성 증가	간단하지만 확장성이 낮음
성능	일반 상황에서 충분히 최적화	구현 실력에 따라 달라짐	고정 크기에서는 단순함
안정성	범위/메모리 관리 부담 감소	메모리/인덱스 실수 가능	범위 초과 위험 큼
추천 상황	대부분의 C++ 코드	자료구조 원리 학습	아주 단순한 고정 크기 데이터

응용: 중복 없는 방문 기록 만들기

여러 번 들어온 값을 set에 넣으면 자동으로 하나만 남습니다.

```
#include <iostream>
#include <set>
using namespace std;

int main() {
    set<string> names = {"Kim", "Lee", "Kim", "Park"};

    if (names.find("Lee") != names.end()) {
        cout << "Lee exists\n";
    }

    for (auto name : names) cout << name << '\n';
}
```

처음 배울 때 자주 하는 실수

- 인덱스로 접근할 수 없음
- 중복 개수가 필요하면 multiset 또는 map 사용
- 정렬이 필요 없으면 unordered_set이 더 빠를 수 있음

비슷한 항목과 차이

- unordered_set은 정렬 없이 평균 $O(1)$ 검색
- multiset은 중복 허용
- map은 key와 value를 함께 저장

한 줄 정리: set은(는) '중복 제거' 상황에서 먼저 떠올리면 좋은 STL입니다.